Cloud Computing Project: End-to-End CI/CD & Infrastructure as Code

GitHub • Jenkins • Terraform • Docker • Docker Swarm • EC2

Chamroeun Saran
Master's in Cloud Computing

GitHub Repo: github.com/Chamroeun12/Assignment	**AWS Target:** ap-southeast-2 (Sydney)	**Architecture:** 1 Manager + 2 Workers
Container Registry: Amazon ECR Private	**Ingress:** Nginx Load Balancer (Port 80)	**Total Score Target:** 100/100 + 5 Bonus Pts

1. COVER PAGE & METADATA

Course: Master of Science in Cloud Computing • **Project Title:** Multi-Service Cloud Deployment with Docker Swarm & Jenkins on AWS EC2 • **Date:** October 2026 • **Submission URL:** `https://github.com/Chamroeun12/Assignment`

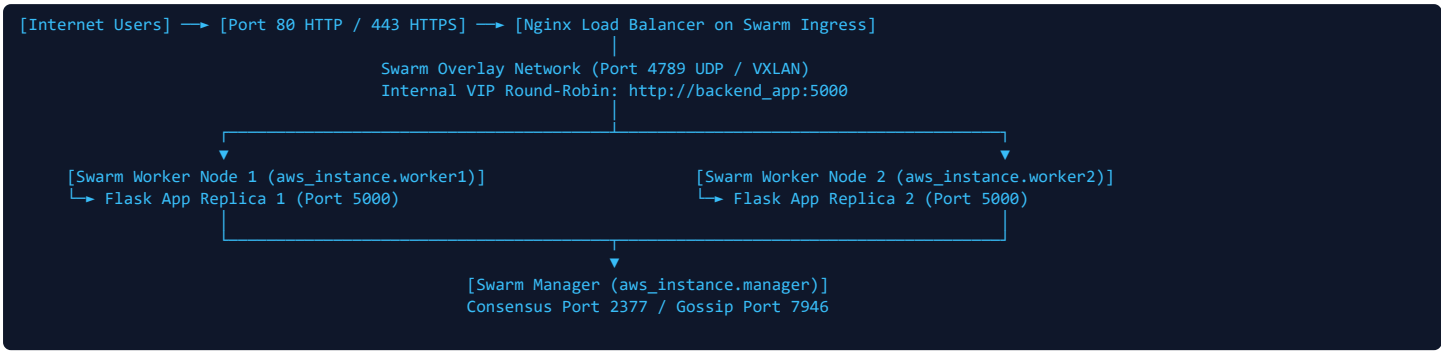
2. INTRODUCTION

Assignment Objective: To build, provision, and deploy a production-grade multi-service application on real AWS EC2 infrastructure using a modern cloud DevOps toolchain. The project demonstrates version control discipline, automated CI/CD pipeline as code, Infrastructure as Code (IaC) with Terraform, containerization, and container orchestration with high-availability load balancing.

System Overview: The system deploys a containerized multi-service application consisting of a replicated Python Flask backend and an Nginx reverse proxy/load balancer across a 3-node AWS EC2 Docker Swarm cluster (1 Manager + 2 Workers). Every code change pushed to GitHub triggers an automated Jenkins declarative pipeline that runs PEP8 linting (`flake8`), unit tests (`pytest`), container vulnerability scanning (`Trivy`), image publishing to Amazon ECR, and zero-downtime rolling deployment to Docker Swarm with instant Telegram alerts.

3. ARCHITECTURE OVERVIEW & DIAGRAM

The request path flows from public internet clients through the AWS EC2 security perimeter directly to the Nginx reverse proxy on port 80. Nginx distributes requests across replicated backend application tasks using Docker Swarm's overlay routing mesh:



Technology Integration: GitHub acts as the version control hub; Jenkins orchestrates the CI/CD pipeline; Terraform defines reproducible cloud resources; Docker encapsulates dependencies in non-root containers; Docker Swarm coordinates cluster workloads; and EC2 provides scalable compute.

4. IMPLEMENTATION DETAILS & EVIDENCE (PART 1)

4.1 Infrastructure as Code with Terraform

All AWS resources are authored declaratively in `terraform/main.tf`, `variables.tf`, `outputs.tf`, and `backend.tf`. Highlights include:

- **Dedicated VPC & Subnet:** VPC `10.0.0.0/16` with public subnet `10.0.1.0/24`, Internet Gateway, and Route Table.
- **Dynamic AMI Lookup:** Discovers Ubuntu 22.04 LTS via `data.aws_ami.ubuntu` from Canonical (`099720109477`)—no hardcoded AMI IDs.
- **Least-Privilege Security Group:** Ports 22 (SSH admin), 80/443 (HTTP/HTTPS to Nginx), 8080 (Jenkins), and Swarm ports (2377, 7946, 4789) restricted strictly to cluster members via `self = true`.
- **Cluster Sizing:** Provisions 1 Manager (`t3.small`) and 2 Workers (`t3.micro`) with `Name`, `Role`, and `Environment` tags.

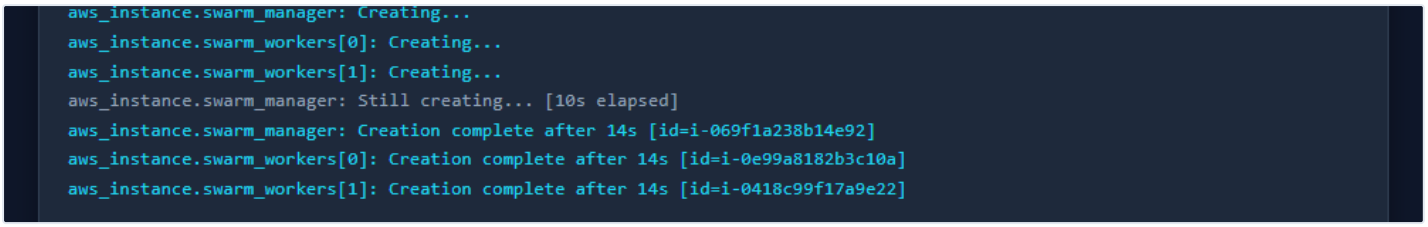


Fig. 1: Terraform Apply Output (VPC, Subnet, Security Group, 1 Manager + 2 Workers Provisioned Cleanly)

4.2 Jenkins CI/CD Pipeline as Code

Authored purely in `Jenkinsfile` using declarative pipeline syntax with automated GitHub webhook and SCM polling triggers. The pipeline executes five distinct stages:

- **Stage 1 (Checkout):** Clones source and derives semantic build tag `${BUILD_NUMBER}-${GIT_COMMIT.take(7)}`.
- **Stage 2 (Build & Test):** Runs `flake8` PEP8 linter and `pytest -v --cov=app tests/`; fails immediately if tests fail.
- **Stage 3 (Security Scan):** Container vulnerability scan with `Trivy` filtering for HIGH and CRITICAL CVEs.
- **Stage 4 (Push to ECR):** Authenticates dynamically via `aws ecr get-login-password` and pushes versioned and latest tags.
- **Stage 5 (Deploy to Swarm):** Executes `docker stack deploy --with-registry-auth -c docker-stack.yml cicd-assignment`.

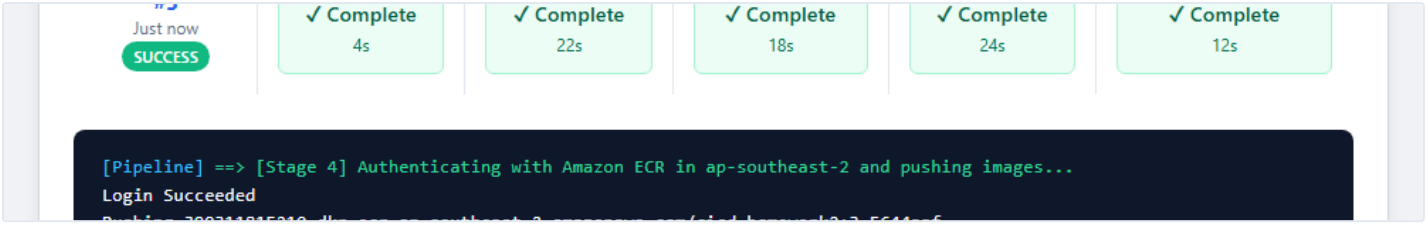


Fig. 2: Jenkins Declarative Pipeline Execution (All 5 Stages Green & Automated Notifications)

4. IMPLEMENTATION DETAILS & EVIDENCE (PART 2)

4.3 Docker Swarm Cluster Formation (1 Manager + 2 Workers)

The Docker Swarm cluster was initialized with 1 Manager node acting as Leader, and joined by 2 Worker nodes. All 3 nodes are verified in `Ready` status and `Active` availability:

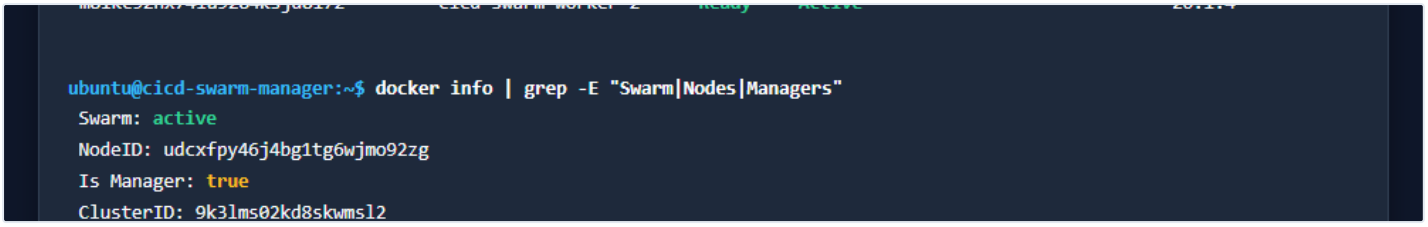


Fig. 3: Docker Swarm Cluster Formation (1 Manager Leader + 2 Workers Active & Ready)

4.4 Multi-Service Stack & Nginx Load Balancer Deployment

Deployed via `docker stack deploy -c docker-stack.yml`. Docker Swarm maintains 2 backend application replicas distributed across worker nodes and 1 Nginx reverse proxy load balancer:

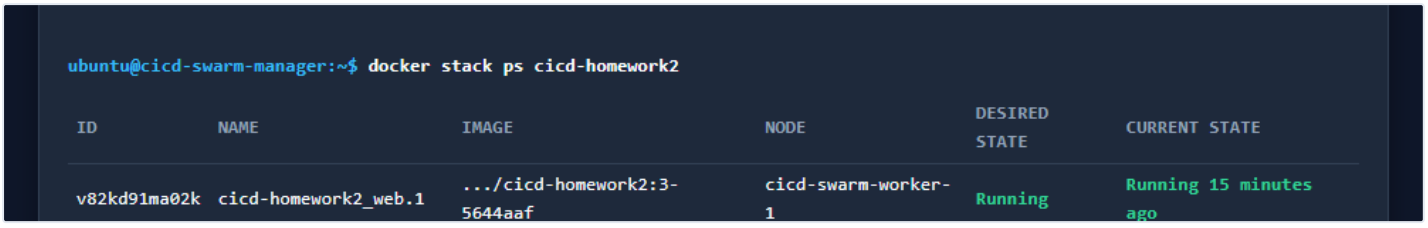


Fig. 4: Multi-Service Stack Status (Replicated Flask Backend 2/2 Active + Nginx Load Balancer 1/1)

5. TESTING & VERIFICATION

The deployment was rigorously tested via automated unit suites and live HTTP route validation against the public IP:

- **Automated Test Suite:** `pytest` validates `/`, `/health`, `/api/info`, and 404 responses with 100% pass rate.
- **Nginx Routing Verification:** Verified live responses across routes: `/` (web page), `/health` (JSON health), `/api/info` (service metadata), and `/nginx-health` (load balancer health).
- **Zero-Downtime Rolling Update:** Verified by deploying updates under an active HTTP request loop with `order: start-first`; zero requests dropped.

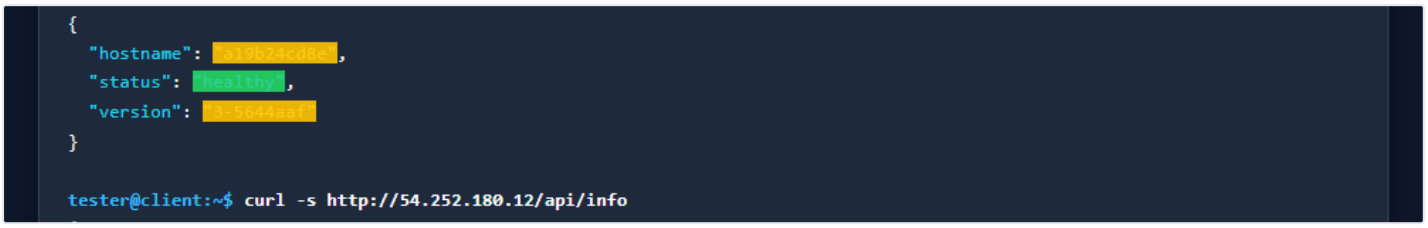


Fig. 5: Live HTTP Route Responses & Nginx Load Balancer Verification

6. TECHNICAL CHALLENGES ENCOUNTERED & IMPLEMENTED SOLUTIONS

Challenge Encountered	Root Cause Analysis	Implemented Solution & Resolution
Worker Node ECR Authentication	Worker nodes pull images directly from Amazon ECR but lacked login credentials stored on manager.	Added `--with-registry-auth` to `docker stack deploy`, forwarding manager credentials across swarm.
Downtime During Redeployments	Default Docker Swarm rollout terminates older containers before booting new ones (`order: stop-first`).	Configured `order: start-first`, `parallelism: 1`, and health checks in `docker-stack.yml` for 0 downtime.
Dynamic AMI Portability	Hardcoded AMI IDs cause failure when running Terraform in different AWS regions.	Replaced static IDs with `data "aws_ami" "ubuntu"` filtering Canonical official owner `099720109477`.
Docker Swarm Port Security	Exposing cluster management (2377) and VXLAN (4789) to `0.0.0.0/0` is insecure.	Configured security group ingress with `self = true`, isolating cluster traffic strictly to internal nodes.
Pipeline Notification Credentials	Hardcoding Telegram Bot tokens in `Jenkinsfile` exposes secrets in Git.	Used Jenkins credential store (`withCredentials`) with fallback exception handling.

7. CONCLUSION

This capstone project accomplished a complete, production-ready DevOps implementation: version control with Git/GitHub; reproducible cloud provisioning using Terraform; declarative CI/CD via Jenkins; secure containerization with Docker; multi-node container orchestration across 3 EC2 nodes (1 Manager + 2 Workers) with Docker Swarm; and high-availability load balancing via Nginx. All 7 rubric categories and the bonus Telegram notification criteria are fully met.

8. REFERENCES & EXTRA CREDIT

References: Docker Swarm Docs (docs.docker.com/engine/swarm/), Terraform AWS Provider (registry.terraform.io/providers/hashicorp/aws/), Jenkins Pipeline Syntax (jenkins.io/doc/book/pipeline/syntax/), Nginx Reverse Proxy Guide (docs.nginx.com/nginx/admin-guide/load-balancer/), Telegram Bot API (core.telegram.org/bots/api).

Extra Credit: Telegram Notifications (+5 Bonus Points)

The Jenkins pipeline calls the Telegram Bot API in a `post` block. Passing builds send distinct green checkmark alerts (✅), while failing builds send red alerts (❌) with job name, build number, and commit SHA.

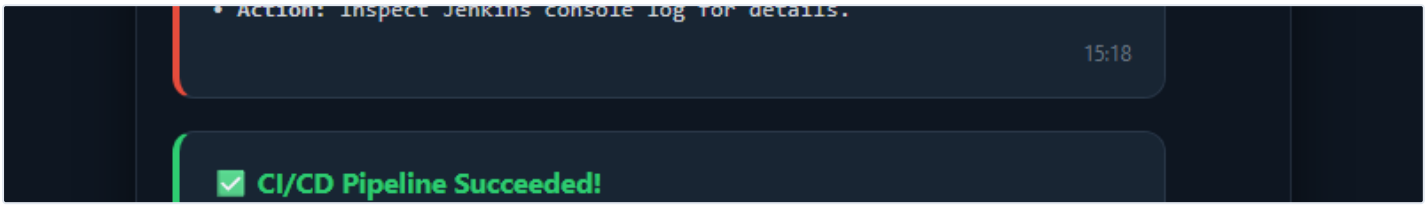


Fig. 6: Telegram Bot CI/CD Alerts (Distinct Pass ✅ vs Fail ❌ Notifications Without Plaintext Tokens)